STM32F429 Harmonic Control Arrays Real-Time Gain Optimizer

GUI User Manual

Document version: 1.1
Date: June 4, 2026

Contents

1 Introduction

The **STM32F429 Harmonic Control Arrays Real-Time Gain Optimizer** is a desktop application for monitoring and tuning a multi-harmonic repetitive controller running on an STM32 microcontroller. It connects to the firmware over a USB virtual COM port (VCP) link, receives high-speed ADC and telemetry data, and provides two complementary methods for computing the complex proportional (K_p) and integral (K_i) controller gains for each harmonic order in real time.

1.1 System Overview

Figure 1: Full application window (connected, tuning in progress).

The application window (Figure 1) is divided into six horizontal zones, from top to bottom:

1. **Connection Bar** — Serial port selection and connect/disconnect controls.
2. **Auto-Tune Bar** — Adam-optimiser-based iterative gain tuner with per-run configuration.
3. **Analytical SysId Bar** — Single-shot plant inversion tuner that measures the plant frequency response and applies the rule $K(h) = k \,/\, G(jh\omega)$.
4. **Status / Gains Bar** — Live readout of tuner status, epoch counter, loss, THD, and per-harmonic complex gains.
5. **Plot Area** — Four real-time plots: ADC waveform, FFT spectrum of ADC, MCU error signal, and tuner convergence curve.
6. **Statistics Bar** — Sample rate, voltage statistics, and packet/drop counters.

1.2 System Requirements

- Python 3.11 or later with `pyserial`, `numpy`, and `matplotlib` installed.
- STM32F429 target flashed with compatible firmware (constants must match `monitor/config.py`).
- USB cable connecting the host PC to the target's VCP.

1.3 Launching the Application

From the project root directory:

```
python run.py
```

or equivalently:

```
python -m monitor
```

2 Connection Bar

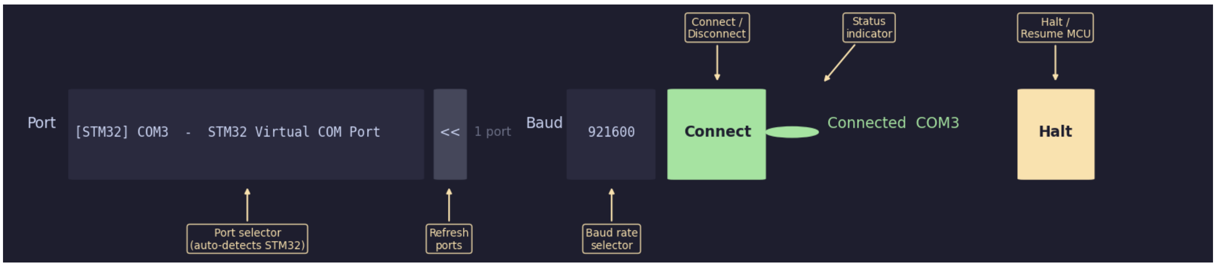

Figure 2: Connection bar with an STM32 device connected on COM3.

The Connection Bar (Figure 2) spans the top of the window and manages the serial link to the firmware.

2.1 Controls

Control	Description
Port drop-down	Lists all available serial ports. Ports belonging to an STM32 device (USB VID `0x0483`) are sorted to the top and prefixed with `[STM32]`. The list shows the device path (*e.g.* `COM3` or `/dev/ttyACM0`) and a human-readable description.
`«` refresh button	Re-scans all serial ports and updates the drop-down. Use this after plugging in the USB cable.
Port count label	Shows the number of detected ports (*e.g.* `1 port`).
Baud drop-down	Selects the serial baud rate. Must match the firmware. Default: **921600**.
`Connect` / `Disconnect`	Toggles the serial connection. On success the button turns red and the port drop-down is disabled.

Control	Description
Status indicator	A coloured dot and text label. Grey = disconnected; green = connected; red = connection failed or lost.
`Halt` / `Resume`	Sends `CMD_SET_MODE` to the firmware. `Halt` stops the controller loop on the MCU (safe state, no output); `Resume` restarts it. The button turns red while halted.

2.2 Step-by-Step: Connecting

1. Plug the USB cable from the STM32 target into the host PC.
2. Click `«` to refresh the port list.
3. Select the `[STM32]` entry from the Port drop-down.
4. Verify the baud rate matches the firmware (default 921 600).
5. Click `Connect`. The status dot turns green and the plots begin updating.

Note

If no `[STM32]` entry appears, check that the USB VCP driver is installed and that the cable supports data transfer (not charge-only).

3 Auto-Tune Bar (Adam Optimiser)

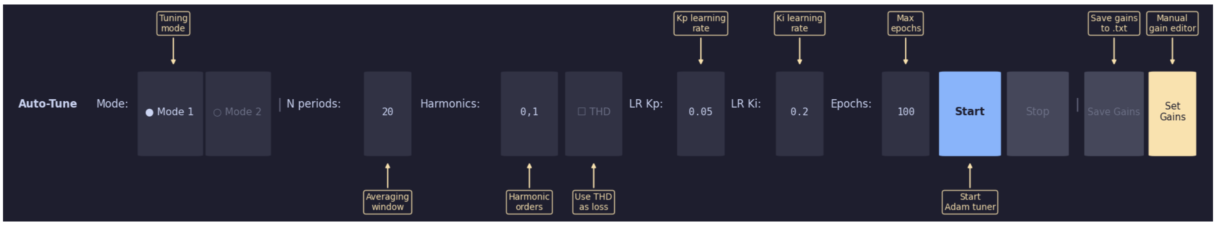

Figure 3: Auto-Tune bar with all controls labelled.

The Auto-Tune Bar (Figure 3) runs an iterative closed-loop optimisation using the **Adam** gradient-descent algorithm. Each epoch collects one measurement window from the MCU, computes the per-harmonic error phasors $E_k = \mathcal{F}\{\text{err}\}\big|_{kf_1}$, and applies the update rule

$$g_{K_p} = \overline{E_k}, \qquad K_p \mathrel{+}= \alpha_{K_p}\, w_k\, \hat{m}_{K_p} / (\sqrt{\hat{v}_{K_p}} + \varepsilon)$$

with first and second moment accumulators ($\beta_1 = 0.9$, $\beta_2 = 0.999$). K_i receives a frequency-rotated gradient $g_{K_i} = g_{K_p} \cdot (-j\,\omega_k)$.

3.1 Controls

Control	Description
Mode 1 / **Mode 2** radio	Selects the per-epoch measurement protocol. **Mode 1**: before each measurement the MCU is halted (`SYS_MODE_STOP`), stale samples are flushed, and the MCU is restarted. Eliminates inter-epoch transient contamination at the cost of a brief output interruption each epoch. **Mode 2**: the MCU continues running throughout. A discard window is applied after each update to absorb transients. Faster but more sensitive to settling time.
N periods field	Number of fundamental-frequency periods to average per epoch. Increasing this reduces noise at the cost of longer epochs. Default: **20**.
Harmonics field	Comma-separated harmonic orders to tune, e.g. `0,1` for DC and the fundamental, or `1,2,3` for the first three harmonics. Order 0 tunes the DC offset channel.
THD checkbox	When checked, the scalar loss function becomes THD %/100 instead of RMS error. Only active when the fundamental amplitude exceeds 10 mV.
LR Kp field	Adam learning rate for K_p. Typical range: 0.01–0.2. Default: **0.05**.
LR Ki field	Adam learning rate for K_i. Typical range: 0.05–0.5. Default: **0.2**.
Epochs field	Maximum Adam iterations before the tuner stops automatically. The tuner can also be stopped at any time with `Stop`. Default: **100**.
`Start`	Begins a new tuning run. Requires an active connection. Adam moment accumulators are reset; previously converged gains are preserved as the starting point.
`Stop`	Halts the tuner after the current epoch and sends `SYS_MODE_RUN` to the firmware. Accumulated gains are not discarded.
`Save Gains`	Opens a file-save dialogue and writes current gains to a `.txt` file (see Section 8.2).
`Set Gains`	Opens the manual gain editor dialogue (see Section 8.1).

3.2 Saturation Guard

If the ADC waveform comes within 5 % of either supply rail, the epoch is flagged `ADC saturated – backing out!` and the gains are automatically reduced by 10 % before the next epoch.

ADC Saturation Warning

Saturation produces clipped error phasors that corrupt the Adam gradients and can cause runaway gain growth. If this message appears repeatedly, reduce the learning rates or the harmonic amplitudes.

3.3 Step-by-Step: Running an Auto-Tune Session

1. Ensure the device is connected (Section 2).
2. Enter the harmonic orders in the **Harmonics** field (e.g. `0,1`).
3. Set **N periods**, learning rates, and epoch count.
4. Select **Mode 1** (safer) or **Mode 2** (faster).
5. Click `Start`. The Status bar updates each epoch; the Convergence plot fills in.
6. When satisfied, click `Stop` or let it complete.
7. Click `Save Gains` to export the result.

4 Analytical SysId Bar

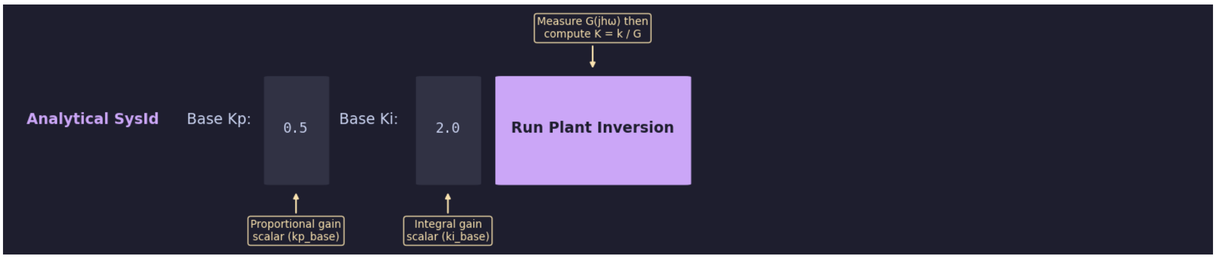

Figure 4: Analytical SysId bar.

The Analytical SysId Bar (Figure 4) provides a faster, non-iterative alternative to the Adam tuner. It automatically measures the plant frequency response $G(jh\omega)$ at each requested harmonic and computes gains using:

$$K_p(h) = \frac{k_p}{G(jh\omega)}, \qquad K_i(h) = \frac{k_i}{G(jh\omega)}.$$

4.1 Measurement Procedure

The identification sequence runs automatically in a background thread:

1. **Zero all gains** and measure the open-loop disturbance phasor $E_{\text{open},k}$ for each harmonic.
2. For each harmonic order h, apply a small test perturbation $K_p = k_{\text{test}}$ and measure the closed-loop phasor $E_{\text{closed},k}$.
3. Compute the plant response:

$$G(jh\omega) = \frac{E_{\text{open},k} - E_{\text{closed},k}}{k_{\text{test}} \cdot E_{\text{closed},k}}$$

4. Reset the perturbation and repeat for the next harmonic.
5. Apply the inversion rule and send gains to the MCU.
6. Measure the post-tuning loss and report it in the Status Bar.

4.2 Controls

Control	Description
Base Kp field	Scalar multiplier k_p applied after plant inversion. A value of 1.0 sets the gain exactly to $1/G$; smaller values provide a stability margin. Default: **0.5**.
Base Ki field	Scalar multiplier k_i for the integral channel. Default: **2.0**.
`Run Plant Inversion`	Starts the SysId sequence. This button and `Start` are both disabled while the sequence runs. Uses the same harmonic order list as the Auto-Tune bar.

Note

The Analytical SysId method and the Adam tuner share the same **Harmonics** field. Verify it is set correctly before running either method.

Warning

The plant identification injects small perturbations into the live control loop. Ensure the system is in a safe steady-state condition before running this procedure.

5 Status and Gains Bar

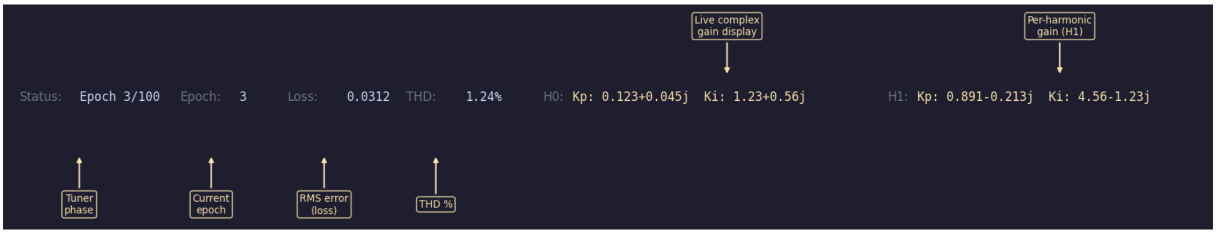

Figure 5: Status / Gains bar during an active tuning run.

Field	Description
Status	Plain-English tuner phase. Examples: `Idle`, `Epoch N/M`, `ADC saturated — backing out!`, `SysId: Pinging Harmonic 1...`, `Converged`, `Stopped`.
Epoch	Current epoch number within the running session.
Loss	Scalar loss for the most recent epoch. With THD off: RMS error in volts; with THD on: THD %/100.
THD	Total Harmonic Distortion of the ADC waveform relative to the fundamental, in percent.
Hn gain display	For each tuned harmonic, shows the current complex K_p and K_i in rectangular form, e.g. `Kp: 0.891-0.213j Ki: 4.56-1.23j`. Updated after each epoch.

6 Plot Area

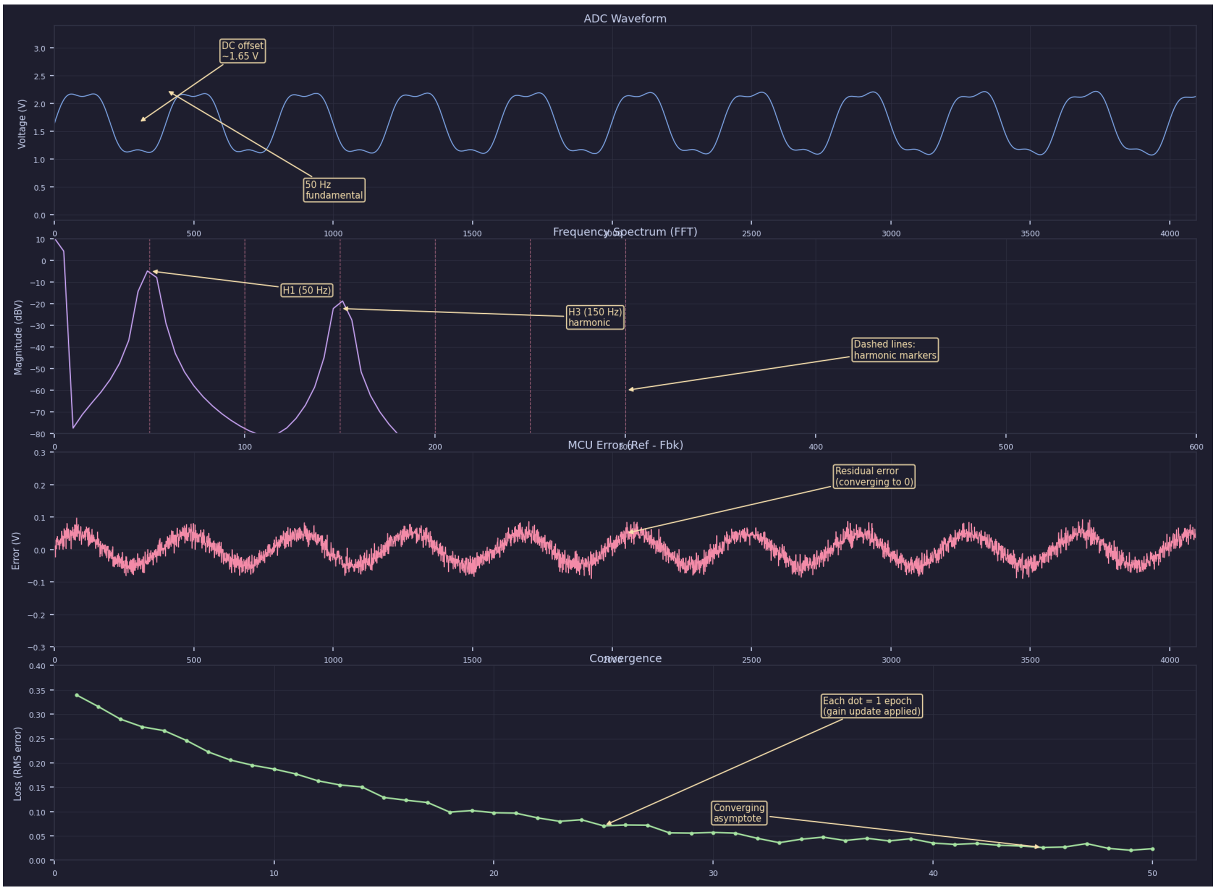

Figure 6: Four live plots with annotated features.

The plot area (Figure 6) contains four subplots updated at approximately 67 Hz (15 ms refresh timer).

6.1 ADC Waveform

Displays the most recent 4096 ADC samples converted to volts ($V = \text{code} \times V_{\text{ref}}/4095$). The y-axis spans 0–3.3 V. Use this plot to confirm the reference signal is present and that the waveform is not clipping.

Note

Telemetry packets use 8-bit ADC encoding; ADC-only packets use the full 12-bit resolution.

6.2 Frequency Spectrum (FFT)

Displays the single-sided magnitude spectrum in dBV, computed with a Hanning window on the current ADC buffer.

Red dashed lines mark the harmonic frequencies $f_1, 2f_1, \ldots, 10f_1$ ($f_1 = 50\,\text{Hz}$). Each marker is annotated with its index `Hk`, amplitude in dBV, and phase relative to the fundamental. The x-axis auto-scales up to half the measured sample rate (capped at 2 kHz).

6.3 MCU Error (Ref − Fbk)

Shows the error signal computed on the MCU (Reference minus Feedback), received via the `TELEMETRY` packet type. The y-axis is fixed at $\pm 1.5\,\mathrm{V}$. As the tuner converges this waveform should shrink toward zero.

Note

This plot requires `PKT_TYPE_TELEMETRY` packets. If only `PKT_TYPE_ADC` packets are received the plot remains flat at zero.

6.4 Convergence

Plots the scalar loss after each completed epoch, one dot per epoch. Both axes adjust dynamically. A healthy run shows a monotonically decreasing curve settling at a small residual value.

Diverging (increasing) loss indicates the learning rate is too high, the plant response is poorly identified, or the ADC is saturating.

7 Statistics Bar

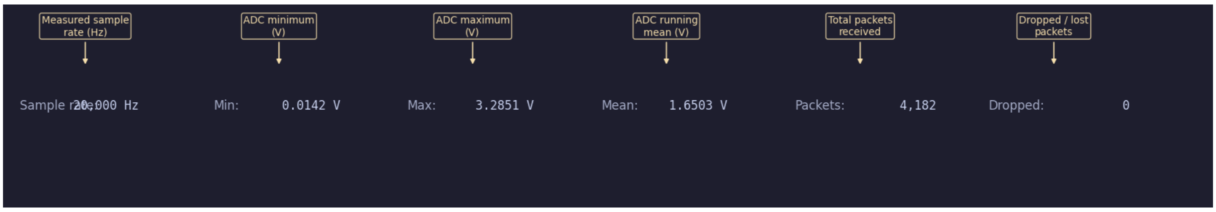

Figure 7: Statistics bar showing live serial link metrics.

Field	Description
Sample rate	Measured incoming sample rate in Hz, averaged over the last second. Should be close to 20 000 Hz.
Min	Minimum ADC voltage in the display buffer (V).
Max	Maximum ADC voltage in the display buffer (V).
Mean	Mean ADC voltage in the display buffer (V). For a 50 Hz signal centred on the supply midpoint this is near 1.65 V.
Packets	Total packets received since connection was established.
Dropped	Dropped samples inferred from sequence number gaps. A non-zero value indicates USB/serial congestion. Try a lower baud rate or reduce host CPU load.

8 Dialogs

8.1 Set Gains Dialog

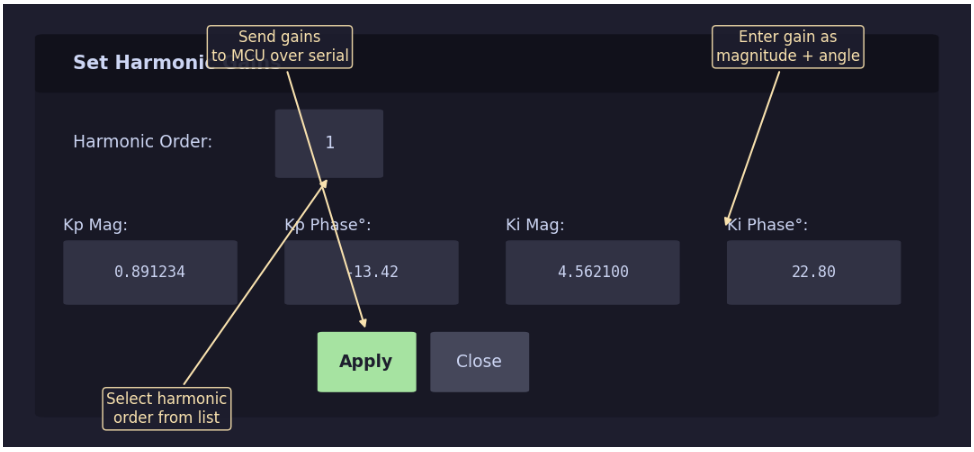

Figure 8: Set Harmonic Gains dialog.

Opened with `Set Gains`, this modal window (Figure 8) allows direct manual editing of K_p and K_i for any tuned harmonic order. Gains are entered in **polar form** (magnitude and phase in degrees).

Control	Description
Harmonic Order drop-down	Selects which harmonic entry to view and edit. Populates the fields automatically when changed.
Kp Mag / **Kp Phase°**	Magnitude and phase (degrees) of the complex K_p gain.
Ki Mag / **Ki Phase°**	Magnitude and phase (degrees) of the complex K_i gain.
`Apply`	Converts polar values to rectangular form and sends them to the MCU via `CMD_SET_HARMONIC`. If no connection is active, gains are stored locally with a warning.
`Close`	Closes the dialog without further action.

Note

To avoid conflicts, stop the tuner before editing gains manually.

8.2 Save Gains to File

Opened with `Save Gains`, writes the current harmonic gains to a plain-text `.txt` file. Output format:

```
Harmonic Controller Gains
Generated : 2025-04-01 14:32:01
Sample rate: 20000.00 Hz
Fund. freq : 50.0 Hz

Order     Kp Real     Kp Imag       |Kp|     Kp Ang deg   ...
-------------------------------------------------------------------
```

```
0        0.123456    0.045678   0.131398       20.34   ...
1        0.891234   -0.213456   0.916296      -13.42   ...
```

Each row contains rectangular and polar representations of both K_p and K_i for one harmonic order.

9 Serial Protocol Reference

All multi-byte integers are little-endian.

9.1 Packet Types

Packet type	Value	Direction / payload
`PKT_TYPE_ADC`	`0x0001`	MCU $\rightarrow$ PC. 12-bit ADC samples (`uint16 LE`).
`PKT_TYPE_CMD`	`0x0002`	PC $\rightarrow$ MCU. Command byte + variable payload.
`PKT_TYPE_TELEMETRY`	`0x0003`	MCU $\rightarrow$ PC. 8-bit ADC (`uint8`) + 16-bit signed error (`int16 LE`), 3 bytes per sample.

9.2 Frame Structure (MCU $\rightarrow$ PC)

```
[0xAA][0x55]          2 bytes   sync
[type  : u16 LE]      2 bytes   PKT_TYPE_ADC or PKT_TYPE_TELEMETRY
[seq   : u32 LE]      4 bytes   sequence number
[count : u16 LE]      2 bytes   number of samples
[payload]                       2 B/sample (ADC) or 3 B/sample (Telemetry)
[crc8  : u8]          1 byte    CRC-8 over all bytes after sync
```

9.3 Command Frame (PC $\rightarrow$ MCU)

```
[0xAA][0x55]          2 bytes   sync
[0x0002 : u16 LE]     2 bytes   PKT_TYPE_CMD
[cmd_id : u8]         1 byte    command identifier
[payload : N bytes]             command-specific payload
[crc8 : u8]           1 byte    CRC-8 over type + cmd_id + payload
```

9.4 Commands

Command	ID	Payload
`CMD_SET_MODE`	`0x01`	1 byte: `0x00` = stop, `0x01` = run.
`CMD_SET_HARMONIC`	`0x02`	1 byte order + 4 $\times$ `float32 LE`: Kp.re, Kp.im, Ki.re, Ki.im.

9.5 CRC-8 Polynomial

Polynomial `0x07`, initial value 0, no reflection (CRC-8/SMBUS):

```
def crc8(data):
    crc = 0
    for b in data:
```

```
        crc ^= b
        for _ in range(8):
            if crc & 0x80:
                crc = ((crc << 1) ^ 0x07) & 0xFF
            else:
                crc = (crc << 1) & 0xFF
    return crc
```

10 Configuration Reference

`monitor/config.py` contains all constants that must match the firmware build.

Constant	Default	Description
`ADC_VREF`	3.3 V	ADC reference voltage.
`ADC_MAX_CODE`	4095	Full-scale ADC code (12-bit).
`DISPLAY_LEN`	4096	Ring-buffer length for display/FFT.
`FUND_FREQ`	50.0 Hz	Fundamental frequency.
`SAMPLE_RATE_NOM`	20 000 Hz	Nominal sample rate.
`SPP`	400	Samples per fundamental period.

Critical Configuration Warning

All constants in `config.py` must exactly match the firmware. A mismatch in `ADC_VREF` or `FUND_FREQ` produces incorrect phasor calculations and prevents convergence.

11 Troubleshooting

Symptom	Resolution
Port list empty after «	Check USB cable (data-capable), VCP driver, target powered on.
Connect failed status	Another application may have the port open. Close it and retry.
Status dot turns red immediately	Serial exception on first read. Verify baud rate and cable.
Waveform plot is flat	MCU may be in `SYS_MODE_STOP`. Click `Resume`.
FFT shows no peak at 50 Hz	Verify `FUND_FREQ` in `config.py` and that the reference signal is active.
Error plot is flat while tuning	Firmware is sending ADC-only packets. Error plot requires `PKT_TYPE_TELEMETRY`.
Dropped counter increasing	Reduce baud rate, close other apps, or lower firmware packet rate.
Convergence diverges	Reduce LR Kp and LR Ki by 2–5×. Check for `ADC saturated` warnings. Try Mode 1 instead of Mode 2.

Symptom	Resolution
`ADC saturated` repeated	Stop the tuner, zero gains via `Set Gains`, restart with smaller learning rates.
`SysId Error` in status bar	Check connection and steady-state MCU operation. See console for full traceback.

12 Quick Reference

Action	How
Connect to device	Refresh (`«`), select port, set baud, click `Connect`.
Start iterative tuning	Set harmonics and parameters, click `Start`.
Stop tuning	Click `Stop` at any time.
Run plant identification	Set Base Kp / Ki, click `Run Plant Inversion`.
Halt MCU output	Click `Halt`; click `Resume` to restart.
Edit a gain manually	Click `Set Gains`, select order, edit, click `Apply`.
Save current gains	Click `Save Gains` and choose a file path.
Disconnect	Click `Disconnect` or close the window.

A Algorithm Notes

A.1 Adam Optimiser (per-harmonic)

The Adam update for harmonic order k with bias correction:

$$
\begin{aligned}
g_t &= \overline{E_k^{(t)}} \quad \text{(conjugate of error phasor)} \\
m_t &= \beta_1 m_{t-1} + (1-\beta_1)\, g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2)\, |g_t|^2 \quad \text{(per-component)} \\
\hat{m}_t &= m_t/(1-\beta_1^t) \\
\hat{v}_t &= v_t/(1-\beta_2^t) \\
K_p^{(t+1)} &= K_p^{(t)} + \alpha_{K_p}\, w_k\, \hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon) \\
w_k &= |E_k|/\sqrt{\textstyle\sum_j |E_j|^2}
\end{aligned}
$$

The per-harmonic weight w_k reallocates the adaptive step in proportion to each channel's contribution to total RMS error, preventing small harmonics from receiving the same absolute update as dominant ones.

A.2 Phasor Extraction

Complex phasors are extracted using a Hanning-windowed correlator:

$$
\begin{aligned}
r &= \frac{2}{N}\sum_{n=0}^{N-1} w[n]\,x[n]\cos\!\left(\frac{\omega_k n}{f_s}\right) \\
s &= \frac{2}{N}\sum_{n=0}^{N-1} w[n]\,x[n]\sin\!\left(\frac{\omega_k n}{f_s}\right) \\
E_k &= r - js
\end{aligned}
$$

The convention $E_k = r - js$ ensures that a pure sine of amplitude A maps to $E_k = -jA$.